Threshold

Architecture Diagram

An AI-directed, multiplayer Fire TV experience

Threshold is a multiplayer (or single player) experience for Amazon Fire TV where The Architect, an AI game master, guides players through a story-driven experience. Players use Fire TV to play and mobile devices as Lens clients to capture and share images. The backend runs on AWS, using Gemini for AI inference, with secure storage, operational monitoring, and remote administration

- Request / Response
- Alternative / Future
- Deployed and Verified
- AWS Service
- Future Implementation

AWS Cloud
(us-east-1)

DuckDNS (External DNS)
Public hostname:
threshold-game.duckdns.org

Resolve to EC2 public IP
32.198.44.211

DNS managed outside AWS

Clients

Fire TV Device (Game Client)
- Runs on Amazon Fire TV /Fire OS(v6 compatible)
- Connects to game sessions over HTTPS
- Renders story, gameplay, and Architect responses

Mobile Device (Lens Client)
- Captures calibration photos
- Uploads images to S3
- Connects to game session over HTTPS
- Shows private tasks

HTTPS
(TLS 1.2+)

Deployed and Verified

Amazon S3
(Calibration Photos)
- Private bucket
- Stores calibration photos
- Lifecycle rule:expire after 1 day

Amazon EC2
(Threshold Backend)
- Caddy (HTTPS reverse proxy)
- Node.js session backend
- systemd (process management)
- CloudWatch agent (log shipping)
- IAM instance role (AWS access)
- SQLite (persistent session/game state)

AWS Systems Manager Parameter Store
- Gemini API key
- Encrypted SecureString parameter
- Least-privilege IAM access

Amazon CloudWatch
(Operational Logging)
- /threshold/session log group (7-day retention)
- Metric filter: calibration.analysis.failed
- Alarm: ThresholdCalibrationFailureAlarm

AI Services

Google Gemini
(Active)
- Gemini API inference
- Calibration image analysis
- AI-powered Architect responses
- Dynamic narrative and puzzle generation

Amazon Bedrock
(Planned / Architected)
- Intended for future use
- Account verification and zero inference quotas (current blocker)
- Architecture prepared for potential integration

Developer Access and Administration

Developer Laptop
- AWS CLI (SSO)
- Remote administration via AWS System Manager
- Deploy application updates
- Monitor logs and alarms

AWS Systems Manager
(Run Command)

AWS Systems Manager
(Run Command)
- Remote EC2 administration without SSH
- Execute commands through AWS CLI
- Deploy backend updates
- Diagnose and manage services

Future Implementation

Infrastructure as Code
- AWS CloudFormation template
- Reproducible infrastructure
- Parameterized configuration
- IAM roles and permissions
- CloudWatch and S3 configuration

Notifications
- Amazon SNS notification topic for alarms
- Email or other notification channels
- Real-time alerts for calibration failures

Enhanced AI Integration
- Enable Amazon Bedrock when account verification and quotas allow
- Support for Nova and other models

Additional AWS Services
- Amazon DynamoDB (persistent gaming state)
- Amazon Cognito (optional authentication)
- Multi-region deployment (future scalability)